

Static Code Analyser

Nico Strohbach

University of Stuttgart

July 14, 2026

Why Taint Analysis?

Idea

Track the flow of untrusted input through the program and determine whether it reaches security-critical operations (sinks).

Why it is suitable

- Detects vulnerabilities based on data flow.
- Can identify indirect flows through assignments and function calls.
- Supports multiple vulnerability classes using the same analysis framework.

Manifest → File Loading → Function Summaries → Taint Analysis →
Sink Detection → Exploit Generation

- Cross-file analysis using function summaries
- Dedicated analyzers for each vulnerability class
- Automatic proof-of-concept (PoC) generation

Taint Analysis

Sources

- `sys.argv`
- `argparse.parse_args()`

Taint Propagation

- Assignments
- Function summaries
- Return values
- String concatenation and f-strings

```
1      path = f"/tmp/{user_input}"  
2      query = "SELECT " + user_input
```

- Lists, tuples and dictionaries

```
1      data = [user_input]  
2      config = {"file": user_input}
```

Sanitizer Detection

Goal

Prevent false positives by recognizing operations that sanitize user-controlled input.

Known Dangerous Patterns

Vulnerability	Exploit Pattern
Path Traversal	"../", "..\\\"
Command Injection	"&&"
Code Injection	"print()"
Unsafe Deserialization	"class"
SQL Injection	Not based on constant patterns, more difficult

Sanitizer Handling

- Detect common sanitization operations:
 - `str.replace()`
 - `str.remove()`
- If dangerous tokens are removed or replaced:
 - Taint propagation can be suppressed

Problem

Tainted values may influence control flow instead of directly reaching a sink.

Approach

- Build an intraprocedural Control Flow Graph (CFG)
- Compute control dependencies
- If a branch condition is tainted:
 - Variables assigned inside dependent blocks become implicitly tainted

- Five dedicated sink analyzers
- Shared infrastructure:
 - Import and alias resolution
 - `from ... import ...` handling
- Findings are passed to an exploit router
- Vulnerability-specific generators create executable Python PoCs

Evaluation Results: Toy Examples

Target	Location	Vulnerability
toy-command-runner	toybackup/runner.py:19	Command Injection
toy-rule-engine	rule_engine/evaluator.py:11	Code Injection
toy-tiny-sql-app	tiny_sql_app/search.py:26	SQL Injection
toy-note-server	note_server/files.py:12	Path Traversal
toy-pickle-profile-loader	profile_loader/loader.py:13	Unsafe Deserialization

Evaluation Results: Real-World Projects

Target	Location	Vulnerability
markdown2	lib/markdown2.py:4800	Command Injection
markdown2	lib/markdown2.py:4750	Path Traversal
markdown2	lib/markdown2.py:4797	Path Traversal
tgconvertor	TGConvertor/___main___py:293	Path Traversal
backup-py	backup.py:655	Command Injection
backup-py	backup.py:696	Command Injection
backup-py	backup.py:623,636,732,768	Path Traversal
backup-py	backup.py:769	Unsafe Deserialization
fossier	src/fossier/cli.py:633	Code Injection
fossier	src/fossier/trustdown.py:44	Path Traversal

Limitations: Exploit generation

```
import sys, os

tainted_variable = sys.argv[2]

if tainted_variable == "taint":
    evil_command = "&& rm -rf *"
    os.system(evil_command)
```

Problem

- The flow to the sink is only influenced by the user input indirectly
- The analyzer can not determine how to reach the sink with a dangerous command

Result

- Exploit is detected, but script does not know the explicit flow
- Exploit script is unprecise

Challenge

SQL injection cannot be reliably detected using simple constant patterns.

Why this is difficult

- SQL syntax is highly context-dependent
- Many different exploit patterns exist
- Sanitization may be incomplete or bypassable